CS4355/6355: Cryptanalysis and DB Security
Instructor: Kalikinkar Mandal
Faculty of Computer Science
University of New Brunswick

Student Name: _____ Matriculation Number: _____

The marking for each task is shown in [ ], and [100] constitutes the full mark.
You must implement the tasks on your own. You can use cryptography library APIs that directly implement ECDSA algorithms as a whole or take some help from ChatGPT. Please submit the fully function source codes via D2L. No marks will be given if your code fails to run or does not produce correct results.

A1. [**60**] In this question, your task is to sign a firmware with file extension `.img` using the ECDSA (Elliptic Curve DSA) algorithm from the cryptography library/package APIs available in your chosen programming language (e.g., `cryptography` in Python or `java.security` in Java). Use the NIST P-256 elliptic curve and SHA3 hash function with 256-bits output in your implementation. Randomly generate 2 KB binary data and store in the firmware file named "`crypto.img`". Your implementation should have all three ECDSA routines namely key generation, sign and verify and should be able to handle signing and verifying the firmware file `crypto.img`.

Sample I/O:

```
----------------------------
KeyGen:
ECDSA signing key x = _____
ECDS verification key vk = _____
----------------------------
Signing:
Hash of the crypto.img file = _____
Signature σ = _____
----------------------------
Verification:
Verification result:  _____
----------------------------
```

A2. [**40**] In this question, your task is to implement the RSA digital signature algorithm with exponent $e = 65537$. Use the SHA3 hash function. Like Question 1, your implementation should have all three routines namely key generation, sign and verify, and should be able to handle signing and verifying the same firmware `crypto.img`. You can reuse the RSA implementation with the same parameters from Programming Assignment 1. Record the time taken by ECDSA and RSA (in milliseconds) and explain which one is faster in terms of computation.

Sample I/O:

```
----------------------------
KeyGen:
```

```
RSA signing key d = _____
RSA verification key vk = (e, n) = _____
----------------------------
Signing:
Hash of the crypto.img file = _____
Signature σ = _____
----------------------------
Verification:
Verification result:  _____
----------------------------
```

Resources for implementations. Below are some libraries in C, Python, Java that you can use for large number operations.

- The GMP library. `https://gmplib.org/` (for C)
- The gmpy2 library. `https://pypi.org/project/gmpy2/` (for Python)
- The BigInteger class in Java

Where can you find ECDSA, SHA256/SHA3 APIs?

Programming language	AES API
C/C++	`OpenSSL`
Python	`cryptography`
Java	`javax.crypto`